

0.70.4. Plus fluide : la lettre au pixel près — en simple

Le 0.70 en une ligne : `glisse_croix(0, px, py, ALPHABET[2], 1)`. Cette fois, un C, parti du pixel (120, 20).

1. Ce qu'il faut comprendre

C'est le 0.70, en plus simple : les quatre `if` des flèches et le `sprite` sont remplacés par une ligne, `glisse_croix` (le 0.73 la reprend avec le A au milieu).

Une autre lettre, ailleurs : un C (`ALPHABET[2]`), parti du pixel (120, 20), en haut à droite.

La vitesse, 1 : un pixel par image, comme au 0.70.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <glisse_croix>` → [0.70.3. La fonction glisse_croix\(\) — suivre la croix, au pixel près](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1142 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage          62 o ← la console, à l'allumage - la mise en route de la console
glisse_croix()     195 o ← main() - une fonction fournie par le langage
main()             38 o ← démarrage - ta fonction
VBlank             8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage      28 o ← image() - attend l'image suivante
AttendreVBlank     16 o ← démarrage - attend que l'écran soit libre
RangerLutins       11 o ← démarrage - range les 40 lutins hors de l'écran
InstallerTransfert 15 o ← démarrage - installe la copie des lutins en mémoire rapide
CopierTuiles       18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
LireManette        37 o ← bouton() - lit les huit boutons
Tuiles            704 o ← CopierTuiles - la police entière
DonneesTransfert   10 o ← InstallerTransfert - la petite routine qui copie les lutins
```

Lettres gravées : toutes (`sprite()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `EffacerCarte`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `Notes`

Fonctions du langage appelées :

```
sprite()          x 1    50 o sur place, dans glisse_croix()
bouton()          x 4    20 o sur place, dans glisse_croix() + LireManette (37 o)
image()           x 1     6 o sur place, dans main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
glisse_croix() x 1    22 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <glisse_croix>    // un lutin qui suit la croix, au pixel près
#include <ALPHABET>        // les lettres de la police : ALPHABET[2] est le C
```

2. UNE VARIABLE — PX

```
uint8_t px = 120;        // le C, en pixels : en haut à droite
```

3. UNE VARIABLE — PY

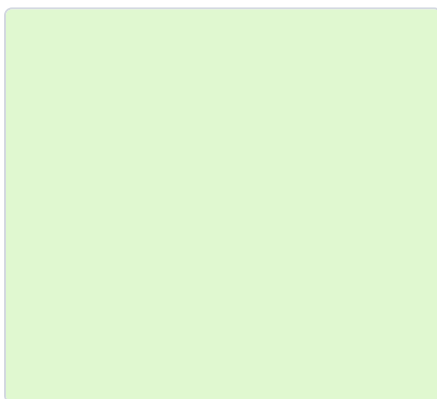
```
uint8_t py = 20;
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

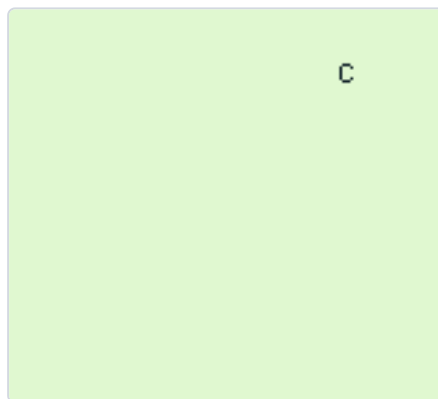
```
int main() {
    while (true) {
        image();
        glisse_croix(0, px, py, ALPHABET[2], 1);    // tout le 0.70, en une ligne
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un C en haut à droite, qui glisse pixel par pixel avec les flèches, comme le A du 0.70.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ 30 pixels en 30 images

